

DSi TECHNICAL INTERVIEW PREPARATION HANDBOOK

Tailored for Md. Tanvir Ahmed Tonmoy

Digital Design | RTL | FIFO/CDC | Timing | FSM | SRAM-CIM | Neuromorphic Computing | RRAM

Built around your actual interview signal

The employee guidance emphasizes digital design basics, FIFO, timing/metastability, Verilog/SystemVerilog, testbenches, FSMs, SRAM/RRAM/crossbars, and ready-valid handshakes. This handbook treats those as the core syllabus, while also preparing you to defend the research items on the CV you submitted.

Updated for your current research direction

SRAM Boolean CIM -> planned MAC extension -> SRAM-array neuromorphic computing
RRAM project -> literature review / modeling preparation stage

September 2026

How to Use This Handbook

This is not meant to be read once like a textbook. It is designed for interview preparation: learn a concept, explain it aloud, draw it from memory, then code or solve a small problem without notes.

Priority	Topics	Target
Tier 1 - Must be automatic	MUX/encoder/counter/register, FFs, blocking vs nonblocking, combinational vs sequential, setup/hold, metastability, FIFO, CDC, ready-valid	Answer and draw without hesitation
Tier 2 - Must be comfortable	Verilog RTL, testbench basics, FSMs, sequence detectors, Gray code, async FIFO flag logic	Write or derive on a whiteboard
Tier 3 - Must defend deeply	6T/8T SRAM, your Boolean CIM paper, Monte Carlo methodology, BNN/XNOR mapping, research-direction change	Explain mechanism, tradeoffs, and your own contribution
Tier 4 - Know enough to discuss honestly	Neuromorphic SRAM direction, RRAM, crossbar VMM, compact-model concepts, PLL project	Conceptual depth without overclaiming progress

Recommended study loop

1. Read one subsection and close the handbook.
2. Explain it in 30-60 seconds as if the interviewer asked you directly.
3. Draw the block diagram or timing path from memory.
4. Write the core RTL without autocomplete.
5. Answer the follow-up question: why is it designed that way?

Interview rule

When you are unsure, do not bluff. State assumptions, reason from fundamentals, and separate what you implemented from what you only studied. For your CV, that distinction is especially important for the MAC and RRAM items.

What your submitted CV makes interviewable

- Research focus: VLSI design, compute-in-memory, memory-based AI accelerators, emerging memories, neuromorphic computing.
- Published SRAM-CIM work: 45 nm 6T SRAM Boolean architecture, AND/OR/NAND/NOR/XOR/XNOR, Monte Carlo delay analysis, hardware-aware BNN evaluation.
- Ongoing-work entries: an SRAM-CIM MAC accelerator and Verilog-A-based RRAM modeling. Your current status has changed, so prepare a precise update rather than defending work you did not complete.
- PLL project: PFD, charge pump, loop filter, VCO, divide-by-48 feedback, locked-state behavior.
- Semiconductor industrial attachment: enough exposure that an interviewer may ask broader digital/physical/analog-flow questions.

1. Interview Map and Your CV Defense Strategy

The strongest preparation strategy is to assume the interview will move from fundamentals to applied design and then into your resume. A candidate who gives clean fundamental answers and is transparent about research status usually creates more confidence than one who gives overcomplicated answers.

Likely Area	Why it matters	Preparation depth
Digital design basics	Directly listed by the employee	Very high
FIFO / CDC	Explicitly called a difference maker	Very high
Setup/hold / metastability	Explicitly highlighted conceptual area	Very high
Verilog / SystemVerilog	RTL code and TB expected	Very high
FSM	Mealy/Moore and sequence detector expected	High
Ready-valid	Explicitly mentioned	High
SRAM/CIM	Directly tied to your strongest publication	Very high
Neuromorphic SRAM	Your current direction; may come up when you update CV status	High
RRAM/crossbar	Explicitly expected because of resume	Medium-high
PLL / other projects	Possible resume follow-up	Medium

1.1 The MAC-to-neuromorphic update

Q: Your CV says SRAM-CIM MAC accelerator. What happened to that project?

A: At the time I submitted the CV, the natural extension of our Boolean CIM work was a scalable MAC-oriented accelerator. Since then, I refined the research direction toward SRAM-array-based neuromorphic computing. The underlying motivation is still memory-centric computing and reducing data movement, but the architecture I am now studying is oriented more toward local synaptic computation and event-driven processing rather than simply extending the Boolean array into a conventional MAC engine.

Q: So did you complete the MAC accelerator?

A: No. I would describe it as the planned extension at the time the CV was submitted, not as a completed accelerator. I prefer to separate completed work from the direction I was investigating.

Q: Have you completed a neuromorphic accelerator?

A: Not yet as a complete accelerator. My current focus is on the architecture and how SRAM arrays can support synaptic storage, event-driven computation, accumulation, and neuron-state operations.

Why this answer works

You acknowledge the CV, give a technically coherent evolution, and avoid pretending that planned work was completed. Research directions changing after a CV submission is normal; overclaiming implementation is not.

1.2 The RRAM status answer

Q: Tell me about your Verilog-A RRAM model.

A: The RRAM work is still in the literature-review and modeling-preparation stage. I am studying the device behavior a compact model needs to capture, including HRS/LRS, SET/RESET behavior, state evolution, variability, and conductance representation. I have not yet produced a validated Verilog-A model, so I would not claim that part as completed implementation.

Q: Then why is it on the CV?

A: It was listed as ongoing work because that was the research activity I had started. The technical progress so far is literature review, device-model understanding, and defining the modeling direction.

1.3 Truth matrix for your CV

Topic	What you can confidently claim	What not to claim
SRAM Boolean CIM	Designed/simulated 45 nm Boolean CIM and evaluated variation-to-BNN impact	Do not reduce it to only a software project
BNN evaluation	Hardware-aware inference/error injection linked to timing variation	Do not claim fabricated silicon
MAC accelerator	Was the intended extension when CV was submitted	Do not claim a finished scalable MAC accelerator
Neuromorphic SRAM	Current research direction and architecture exploration	Do not claim a completed neuromorphic chip unless true
RRAM	Literature review, compact-model study, device/crossbar concepts	Do not claim a validated Verilog-A model
PLL	Cadence design/simulation of the listed loop blocks and lock behavior	Do not overstate silicon validation

2. Digital Design Fundamentals

Your goal is not just to know definitions. You should be able to move between truth tables, Boolean expressions, block diagrams, RTL, and timing behavior.

2.1 Combinational vs sequential logic

Combinational	Sequential
Output depends on present inputs	Output depends on present inputs and stored state
No state element required	Uses latches/flip-flops/registers or other state
Examples: MUX, decoder, encoder, adder	Examples: counter, register, FSM
Often described by Boolean equations	Described by next-state and output equations
In RTL: assign / always @(*) / always_comb	In RTL: edge-triggered always / always_ff

Q: What makes a circuit sequential?

A: Stored state. A clock is common in synchronous sequential logic, but the defining property is that the current behavior depends on previous state.

2.2 Latch vs flip-flop

- Latch: level-sensitive. While enable is active, the output can follow the input.
- Flip-flop: edge-triggered. The input is sampled around a clock edge.
- Unintended latches appear when combinational RTL does not assign an output in every path.

```
always @(*) begin
    if (en)
        y = a;      // y must remember its old value when en=0 -> latch
end

always @(*) begin
    y = 1'b0;      // default assignment prevents latch
    if (en)
        y = a;
end
```

Q: Why are accidental latches usually undesirable?

A: They introduce level-sensitive timing behavior, complicate timing closure and verification, and are often not what the RTL designer intended.

2.3 D, T, JK and SR flip-flops

FF	Characteristic behavior	Typical use / interview point
D	$Q(\text{next})=D$	Primary state element in synchronous RTL
T	$T=0$ hold, $T=1$ toggle	Counters / conceptual derivation
JK	00 hold, 01 reset, 10 set, 11 toggle	Generalized SR without simple forbidden 11 state
SR	Set and reset controls	Know invalid/forbidden state depends on implementation

Q: Why is DFF dominant in RTL design?

A: Because next-state combinational logic can calculate the next value and the DFF simply stores it at the clock edge. This maps naturally to synchronous design methodology.

2.4 MUX, decoder, encoder and priority encoder

- A 2^n -to-1 MUX needs n select bits.
- A decoder maps an n -bit code to one of 2^n outputs.
- An encoder maps one-hot input information to a binary index.
- A priority encoder resolves the case where several inputs are asserted simultaneously.

4-input priority: $D3 > D2 > D1 > D0$

$D = 0111 \rightarrow D2 \text{ wins} \rightarrow \text{encoded output } 10$

Q: Why is a normal encoder ambiguous if multiple inputs are high?

A: Because more than one binary code would be valid. A priority rule removes the ambiguity.

3. Whiteboard Combinational Design

These are high-probability whiteboard questions because they test hierarchy, Boolean reasoning, and how quickly you can organize a circuit.

3.1 Build an 8:1 MUX from 2:1 MUXes

```

Stage 1 (S0): 4 MUXes
D0,D1 -> A0
D2,D3 -> A1
D4,D5 -> A2
D6,D7 -> A3

Stage 2 (S1): 2 MUXes
A0,A1 -> B0
A2,A3 -> B1

Stage 3 (S2): 1 MUX
B0,B1 -> Y

```

Total number of 2:1 MUXes = $4 + 2 + 1 = 7$. The tree depth is three MUX stages, so if each MUX has delay t_{MUX} , the idealized critical-path delay is about $3 \cdot t_{MUX}$.

Q: Generalize this to a $2^n:1$ MUX.

A: A balanced binary tree uses $2^n - 1$ two-input MUXes and has n stages of depth.

Q: Why use a tree instead of a long cascade?

A: A balanced tree minimizes logic depth, which reduces critical-path delay compared with a serial chain.

3.2 Priority encoder using MUXes

For a 4-to-2 priority encoder with D3 highest priority:

```

Y1 = D3 + D2
Y0 = D3 + (~D2 & D1)
Valid = D3 + D2 + D1 + D0

```

One way to think in MUX form is to ask: if the highest-priority signal is asserted, what should the output be? Otherwise, fall through to the lower-priority logic. That naturally creates a MUX chain or priority tree.

Q: What would you draw first?

A: I would state the priority order, write the truth behavior for the highest asserted input, then derive each output bit or construct a MUX chain that selects the code of the highest asserted input.

3.3 Universal MUX reasoning

A useful interview trick is to use the select input as a Boolean variable and wire the data inputs to 0, 1, another variable, or its complement. A 2:1 MUX implements:

```
Y = (~S & D0) | (S & D1)
```

Because of this, MUXes are functionally complete building blocks when constants and complemented signals are available.

4. Counters, Registers and Shifters

These questions look easy, but interviewers often use them to reveal whether you understand synchronous state updates, enable behavior, reset semantics, and width overflow.

4.1 Synchronous vs asynchronous counters

Ripple / asynchronous counter	Synchronous counter
Only first FF receives the external clock	All FFs receive the same clock
Next stage is clocked by previous FF output	Combinational next-state logic decides toggles
Delay ripples through stages	Outputs update after the same active edge
Simple but slower / glitch-prone intermediate states	Preferred for high-speed synchronous design

Q: Why can ripple counters show temporary incorrect states?

A: Each stage changes after the propagation delay of the preceding stage, so the output vector does not transition atomically.

4.2 Up/down counter RTL

```
module counter #(parameter W=4) (
    input clk,
    input rst,
    input en,
    input up,
    output reg [W-1:0] count
);

always @(posedge clk) begin
    if (rst)
        count <= {W{1'b0}};
    else if (en) begin
        if (up) count <= count + 1'b1;
        else   count <= count - 1'b1;
    end
end
endmodule
```

Q: What happens when en=0?

A: No assignment occurs in the clocked branch, so the flip-flops retain their previous value. This is intentional state holding, not latch inference.

Q: What happens when a 4-bit counter at 1111 increments?

A: With ordinary unsigned 4-bit arithmetic, the result wraps to 0000 because the carry-out is discarded.

4.3 Shift registers

- SISO: serial in, serial out.
- SIPO: serial in, parallel out.
- PISO: parallel in, serial out.
- PIPO: parallel in, parallel out.

```
always @(posedge clk) begin
    if (rst) q <= 4'b0000;
    else   q <= {q[2:0], serial_in};
end
```

Q: What operation does {q[2:0], serial_in} perform?

A: It shifts the existing register toward the MSB side shown by the concatenation and inserts `serial_in` into bit 0. Always verify direction from the indices instead of relying on the words left/right.

5. Timing: Setup, Hold, Skew and Slack

Timing questions are often where an interview moves from classroom logic to real digital IC reasoning. Always draw the register-to-register path first.

```

launch FF  --tCQ--> combinational logic --routing--> capture FF
  ^                               ^
launch clock                               capture clock

```

5.1 Setup time

Setup time is the minimum interval before the capture edge during which the input to the receiving flip-flop must remain stable. A setup violation means the data arrives too late relative to the capture edge.

Approximate setup condition:

$$T_{clk} \geq t_{CQ(max)} + t_{Comb(max)} + t_{Setup} - t_{Skew}$$

$$\text{Setup slack} = \text{required arrival time} - \text{actual arrival time}$$

- Fixes: reduce combinational depth, pipeline, use faster cells, reduce fanout, improve placement/routing, or increase clock period.
- Increasing the clock period lowers frequency and can fix setup, because the next capture edge moves later.

5.2 Hold time

Hold time is the minimum interval after the capture edge during which the input must remain stable. A hold violation means the new data reaches the receiving flip-flop too quickly.

Approximate hold condition:

$$t_{CQ(min)} + t_{Comb(min)} \geq t_{Hold} + t_{Skew}$$

$$\text{Hold slack} = \text{actual minimum delay} - \text{required minimum delay}$$

- Fixes: add delay buffers/cells, route detours, cell sizing, or otherwise increase the minimum data-path delay.
- Reducing the clock frequency normally does not fix hold because hold is checked around the same active edge.

Q: Why can frequency help setup but not hold?

A: Setup is about reaching the next capture edge, so a longer period gives more time. Hold is about preventing the new data from arriving too soon after the current edge, so moving the next edge does not solve that minimum-delay requirement.

5.3 Clock skew

Define skew carefully before using signs. With skew = capture-clock arrival - launch-clock arrival, positive skew means the capture edge arrives later. This generally helps setup but can hurt hold.

Q: Can clock skew be both helpful and harmful?

A: Yes. The same later capture-clock arrival that increases setup time margin can reduce hold margin. Timing closure balances both constraints.

5.4 Combinational glitches and timing

Different logic paths have different propagation delays. During an input transition, temporary incorrect values can appear before the circuit settles. In synchronous design, the key requirement is that the correct value is stable when sampled; however, glitches can still waste dynamic power or cause problems if used as clocks/asynchronous controls.

6. Metastability and Clock-Domain Crossing (CDC)

This section is critical. The employee explicitly highlighted metastability and asynchronous FIFO. A strong answer connects the physical phenomenon to the architecture used to contain it.

6.1 What metastability actually is

A flip-flop has an analog internal decision process. If its input changes inside the setup/hold aperture, the internal nodes can take longer than normal to resolve to a legal logic level. The output may temporarily be neither a clean 0 nor a clean 1 from the perspective of downstream timing.

Q: Can metastability be eliminated?

A: No. The probability that it propagates can be made extremely small, but an asynchronous input can always arrive arbitrarily close to a sampling edge.

6.2 Two-flop synchronizer

```
async signal -> FF1 -> FF2 -> synchronized signal
                ^
                |
                may enter metastability
```

FF2 provides approximately one extra destination-clock period for FF1 to resolve.

A two-flop synchronizer is appropriate for a single-bit level signal that can tolerate synchronization latency. The first flip-flop is the metastability containment stage; downstream logic should use the second stage.

Q: Why not just use one flip-flop?

A: If the first stage becomes metastable, the uncertain output directly reaches functional logic. A second stage gives the first stage additional resolution time and greatly improves MTBF.

6.3 Why a 2-FF synchronizer is unsafe for an arbitrary multi-bit bus

If every bit is synchronized independently, the bits may resolve or be sampled on different destination cycles. The destination can therefore observe a word that never existed in the source domain.

Better choices

For a slowly changing data bus, use a handshake so the source holds data stable while a synchronized control signal crosses. For sustained streaming or buffering between unrelated clocks, use an asynchronous FIFO.

6.4 Pulse crossing

A short source-domain pulse can be completely missed by a slower destination clock. Common techniques include pulse stretching, toggle synchronizers, request/acknowledge handshakes, or an async FIFO when the event carries data.

Q: Suppose a one-cycle pulse from a 200 MHz domain goes to a 50 MHz domain. Is a plain 2-FF synchronizer enough?

A: Not necessarily. The 5 ns pulse may occur entirely between 20 ns destination edges. I would stretch it, convert it to a toggle/event handshake, or use a protocol that guarantees observation.

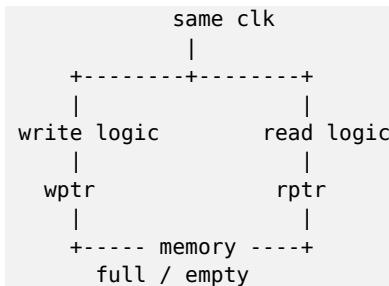
6.5 Reset crossing

A common robust pattern is asynchronous assertion and synchronous deassertion within each clock domain. Immediate assertion can force a safe state, while synchronized deassertion prevents different registers in the same domain from leaving reset on uncontrolled edges.

7. FIFO - The High-Value Difference Maker

Treat FIFO as a complete design topic, not just a queue definition. You should understand architecture, pointers, full/empty detection, wrap bits, asynchronous pointer synchronization, Gray code, and corner cases.

7.1 Synchronous FIFO architecture



The read and write sides share one clock. No CDC is required. The memory may be implemented using registers or inferred SRAM/BRAM depending on depth and technology.

7.2 Pointer width and wrap bit

For a FIFO depth of 2^N , the memory address needs N bits. A convenient pointer implementation uses $N+1$ bits, where the extra MSB records which lap of the circular buffer the pointer is in.

Example: depth = 8 -> address bits = 3 -> pointer bits = 4

Empty: `wptr == rptr`

Full: lower 3 address bits equal AND wrap bits different

Q: Why can `wptr==rptr` not by itself distinguish full from empty?

A: After a complete wrap, the writer can point to the same memory address as the reader even though the FIFO contains a full buffer of unread entries. The extra phase/wrap information distinguishes the cases.

7.3 Safe read and write conditions

```

write_fire = wr_en && !full
read_fire  = rd_en && !empty
  
```

- Overflow: attempting to write when full. Usually block the write and optionally flag an error.
- Underflow: attempting to read when empty. Usually block the read and optionally flag an error.
- Simultaneous read and write in a synchronous FIFO can both occur in the same cycle if conditions allow.

7.4 Asynchronous FIFO



The memory is shared, but control is local to each clock domain. Each domain makes decisions using its own local pointer and a synchronized representation of the remote pointer.

7.5 Why binary pointers are not crossed directly

A binary increment can flip several bits at once. Example: 0111 -> 1000 changes four bits. Because each bit has different routing and clock-to-Q delay, the destination might sample an incoherent intermediate pattern.

Gray code changes only one bit between adjacent counts. This does not remove metastability, but it prevents a normal single-step pointer increment from presenting several simultaneously changing pointer bits to the synchronizer.

```
Binary-to-Gray:
gray = binary ^ (binary >> 1)

Example binary sequence: 000,001,010,011,100
Gray sequence:           000,001,011,010,110
```

Q: Why synchronize Gray pointers instead of binary pointers?

A: Because only one Gray bit changes for each adjacent pointer step, reducing the risk that the destination interprets a multi-bit transition as an incoherent pointer value.

Q: Does Gray code eliminate metastability?

A: No. The changing Gray bit can still be sampled inside the setup/hold window. We still use synchronizer flip-flops. Gray code limits transition ambiguity; synchronizers contain metastability.

7.6 Async FIFO empty condition

On the read side, the FIFO is empty when the next/current read pointer equals the synchronized write pointer, depending on whether the design registers flags from next-state logic. Conceptually, the reader has caught up with everything that has safely crossed into its domain.

```
Conceptual condition:
rgray_next == synchronized_wgray -> empty_next
```

7.7 Async FIFO full condition

The full condition means the next write would advance the write pointer exactly one entire buffer ahead of the synchronized read pointer. In the standard power-of-two Gray-pointer design, this appears as a specific inversion of the leading Gray bits while the remaining bits match.

```
Common form for pointer width N+1:
wgray_next == { ~rgray_sync[N:N-1], rgray_sync[N-2:0] }
```

Do not memorize this blindly. Understand it as: same circular-buffer position, opposite wrap relationship, expressed in Gray code.

Q: Why is full generated in the write-clock domain?

A: Because full controls whether the writer is allowed to advance. The decision must be synchronous to wclk and use a read pointer safely synchronized into the write domain.

Q: Why is empty generated in the read-clock domain?

A: Because empty controls the reader, so it must be synchronous to rclk and use the write pointer synchronized into the read domain.

7.8 FIFO interview corner cases

- Read and write on the same cycle in a synchronous FIFO.
- Write attempt while full; read attempt while empty.
- Pointer wrap-around.
- Reset behavior: both local pointers start at the same logical position so FIFO begins empty.
- Flag latency in an async FIFO: synchronized remote-pointer information is delayed by destination clock cycles, which is intentional for safe CDC.

8. Ready-Valid Handshake and Backpressure

Ready-valid is a compact protocol concept that appears throughout pipelines, streaming interfaces, FIFOs, and on-chip interconnects. The interviewer may test whether you understand transfer semantics rather than memorized signal definitions.

8.1 Core rule

```
transfer occurs on an active clock edge when:
valid && ready == 1
```

valid	ready	Meaning
0	0	No valid data and receiver not advertising capacity
0	1	Receiver is ready, but sender has nothing valid
1	0	Valid item is waiting; receiver applies backpressure
1	1	Transfer occurs on the sampling clock edge

8.2 Stability requirement

If the sender asserts valid but the receiver deasserts ready, the sender must keep the item available. In a conventional ready-valid protocol, data and valid remain stable until a handshake accepts that item.

```
Cycle:      1    2    3    4
valid:      1    1    1    1
ready:      0    0    1    1
data:       A    A    A    B
              ^
              A transfers
```

Q: What is backpressure?

A: The downstream block deasserts ready, forcing the upstream block to stop advancing or to buffer its current item.

Q: Can the sender wait for ready before asserting valid?

A: Some protocols permit behavior that can create deadlock if both sides wait for each other. A common robust rule is that the producer asserts valid whenever it has data, independently of ready, and holds it until accepted.

8.3 Ready-valid and FIFO

A FIFO naturally decouples producer and consumer timing. The producer-side ready is often equivalent to !full; the consumer-side valid is often equivalent to !empty. This is not the only implementation, but it is a useful mental model.

```
input_ready  = !fifo_full
output_valid = !fifo_empty
```

9. Verilog RTL Essentials

For the interview, you should be able to write small RTL blocks without syntax hunting. The goal is correctness first, elegance second.

9.1 Blocking vs nonblocking

Blocking =	Nonblocking <=
Executes procedurally in statement order	RHS is evaluated and updates are scheduled
Use for combinational procedural logic	Use for edge-triggered sequential logic
Useful for intermediate combinational variables	Models simultaneous FF updates

```
always @(*) begin
    y = (a & b) | c;    // blocking for combinational logic
end

always @(posedge clk) begin
    q <= d;            // nonblocking for state
end
```

Q: Why does nonblocking better model a bank of flip-flops?

A: All RHS values are sampled using the old state, and the state updates are committed together. This matches the conceptual behavior of registers triggered by the same edge.

9.2 always @(*)

The simulator automatically includes every signal read by the block in the sensitivity list. This avoids bugs caused by forgetting a signal in a manually written sensitivity list.

Q: What is the SystemVerilog alternative?

A: always_comb, which also expresses the designer intent that the block is combinational and enables stronger tool checks.

9.3 wire, reg and logic

- Verilog wire: net driven continuously, by assign or module connections.
- Verilog reg: procedural variable assigned inside always/initial blocks. The name does not guarantee physical hardware is a register.
- SystemVerilog logic: a general variable type that removes many legacy wire/reg usage distinctions, subject to driver rules.

9.4 case, casez and defaults

```
always @(*) begin
    y = 2'b00;
    valid = 1'b1;
    casez(d)
        4'b1??? : y = 2'b11;
        4'b01?? : y = 2'b10;
        4'b001? : y = 2'b01;
        4'b0001 : y = 2'b00;
        default: begin
            y = 2'b00;
            valid = 1'b0;
        end
    endcase
end
```

casez allows Z and ? wildcard matching. Be cautious with casex because treating X as a wildcard can hide unknown-state bugs during simulation.

9.5 Parameterization

```
module reg_en #(parameter W=8) (  
    input clk, rst, en,  
    input [W-1:0] d,  
    output reg [W-1:0] q  
);  
...  
endmodule
```

Parameterization demonstrates reusable RTL thinking and often earns follow-up questions about width, overflow, and synthesis.

10. Testbench and SystemVerilog Basics

The employee explicitly mentioned simple DUT testbenches, fork-join, data types, case statements, always vs initial, and optional class-based TB concepts.

10.1 Minimal testbench structure

```
`timescale 1ns/1ps
module tb;

  reg clk;
  reg rst;
  reg a, b;
  wire y;

  dut uut (.clk(clk), .rst(rst), .a(a), .b(b), .y(y));

  initial begin
    clk = 0;
    forever #5 clk = ~clk;
  end

  initial begin
    rst = 1; a = 0; b = 0;
    #20 rst = 0;
    #10 a = 1; b = 0;
    #10 a = 1; b = 1;
    #20 $finish;
  end

endmodule
```

Memorize the structure, not the exact code

1) signal declarations, 2) DUT instantiation, 3) clock generation, 4) reset/stimulus generation, 5) checking, 6) finish.

10.2 initial vs always

- initial begins once at simulation start and executes its statements once unless it contains a loop.
- always repeats forever by definition.
- A common clock generator is always #5 clk = ~clk; or initial begin clk=0; forever #5 clk=~clk; end.

10.3 Self-checking testbenches

```
if (y !== expected) begin
  $display("FAIL t=%0t y=%b expected=%b", $time, y, expected);
  errors = errors + 1;
end
```

A self-checking testbench is stronger than manual waveform inspection because the expected behavior is encoded explicitly and regressions can run automatically.

10.4 fork-join

```
fork
  #10 a = 1;
  #20 b = 1;
  #30 c = 1;
join
```

Construct	Parent continues when...
fork...join	all child threads finish
fork...join_any	any one child finishes

Construct	Parent continues when...
fork...join_none	immediately, without waiting for children

10.5 Class-based verification - enough to discuss

Generator -> Transaction -> Driver -> DUT -> Monitor -> Scoreboard

- Transaction: object representing stimulus fields and expected/observed data.
- Generator: creates transactions, often randomized.
- Driver: converts transactions into pin-level activity.
- Monitor: samples DUT interface activity.
- Scoreboard: compares actual behavior with a reference model / expected behavior.
- Mailbox/queue: common communication mechanisms between components.

Q: Do you know UVM?

A: A safe answer if accurate: I understand the basic class-based verification architecture and the roles of transaction, generator, driver, monitor and scoreboard, but I have not yet worked deeply with full UVM methodology.

11. Finite-State Machines (FSM)

FSM questions test whether you can convert a verbal specification into states, transitions, outputs, and synthesizable RTL.

11.1 Moore vs Mealy

Moore	Mealy
Output depends only on current state	Output depends on current state and current input
Often more states	Often fewer states
Outputs change with registered state changes	Output can react combinational to input
Simpler output timing	Potentially lower response latency but can glitch if inputs are not controlled

Moore: $Y = f(\text{state})$

Mealy: $Y = f(\text{state}, \text{input})$

11.2 Sequence detector methodology

For target 1011, define each state by the longest prefix of the sequence already matched:

S0: nothing useful matched

S1: matched 1

S2: matched 10

S3: matched 101

Detection occurs when the next expected 1 arrives.

For overlapping detection, after a match you do not automatically return to S0. You transition to the state representing the longest suffix of the received stream that is also a prefix of the target sequence.

Q: Why is overlap important?

A: Because the final bits of one detected sequence may already be the beginning of the next sequence. Throwing them away misses valid detections.

11.3 Robust RTL template

```
always @(posedge clk) begin
    if (rst) state <= S0;
    else     state <= next_state;
end

always @(*) begin
    next_state = state;
    case (state)
        S0: if (x) next_state = S1;
        S1: ...
        default: next_state = S0;
    endcase
end

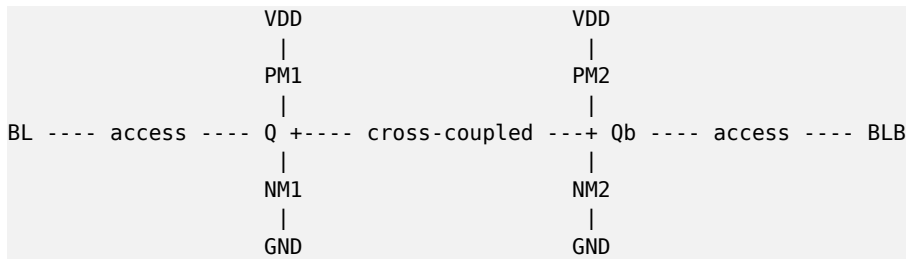
always @(*) begin
    y = 1'b0;
    case (state)
        ...
    endcase
end
```

The default assignment `next_state = state` prevents accidental latches and makes unspecified transitions hold state. A default case provides recovery from illegal state encodings.

12. SRAM Fundamentals You Must Own

Your resume contains serious SRAM-CIM work. Expect interviewers to test whether you understand the memory cell itself before discussing compute-in-memory.

12.1 6T SRAM anatomy



Both access transistors are controlled by WL.

Four transistors form two cross-coupled CMOS inverters, creating a bistable latch. Two access NMOS transistors connect the internal nodes to the complementary bitlines when WL is asserted.

12.2 Hold operation

WL is low, the access devices are off, and the cross-coupled inverters regenerate the stored state. Static leakage still exists, but no refresh is needed as long as supply power remains.

12.3 Read operation

6. Precharge BL and BLB, usually high and equalize them.
7. Assert WL, connecting internal storage nodes to bitlines.
8. The side storing 0 creates a discharge path through the pull-down/access path.
9. A small bitline differential develops.
10. A sense amplifier resolves the small differential to a full-swing logic value.

Read disturb occurs because the internal 0 node is no longer completely isolated. Its voltage can rise through the access device. Cell sizing must preserve the stored state during this disturbance.

12.4 Write operation

The write driver forces complementary bitline values. When WL rises, the write path must overpower one side of the cross-coupled latch sufficiently to flip the state. This produces a design tradeoff with read stability.

12.5 Cell ratio and writeability

- For read stability, the pull-down device should be sufficiently strong relative to the access device so the stored 0 is not raised above the inverter trip point.
- For writeability, the access/write path must be strong enough relative to the pull-up PMOS to force a new value.
- These requirements compete, so transistor sizing is a central 6T SRAM design tradeoff.

12.6 Static Noise Margin (SNM)

SNM measures the maximum DC noise disturbance the bistable cell can tolerate without changing state. It is commonly visualized using the butterfly curve created from the voltage-transfer characteristics of the two cross-coupled inverters. Read SNM is typically lower than hold SNM in a conventional 6T cell because the read path perturbs the internal node.

Q: What does a larger SNM mean?

A: More voltage-noise tolerance before the stored state becomes unstable.

13. 8T SRAM, CIM and Your Published Work

13.1 Why add a separate read port?

A common 8T SRAM adds two transistors to create a read path controlled separately from the 6T storage/write core. The read bitline observes the stored value without directly connecting the storage node to the large read-bitline capacitance.

6T SRAM	8T SRAM with decoupled read port
Smaller cell area	Larger area
Read operation disturbs storage node	Read path can be isolated from storage node
Good density	Improved read stability / low-voltage robustness
CIM possible but read/write constraints matter	Extra read path can be useful for specialized compute structures

13.2 Why compute-in-memory?

Conventional:

Memory -> bus/interconnect -> compute -> bus/interconnect -> memory

CIM:

Memory array + local compute / sensing / accumulation

For data-intensive workloads, arithmetic is not the only energy cost. Repeatedly transporting weights and activations between memory and compute units consumes time and energy. CIM tries to reduce this movement by performing useful operations at or near the memory array.

Q: Is CIM automatically faster and lower power?

A: No. It can reduce data movement, but peripheral circuitry, sensing, ADC/DACs, precision requirements, array parasitics, and control overhead can dominate. The benefit depends on the architecture and workload.

13.3 Your Boolean SRAM-CIM story

Your strongest resume story should be explained as a cross-layer chain:

```

45 nm SRAM-CIM circuit
|
Boolean operations: AND / OR / NAND / NOR / XOR / XNOR
|
Monte Carlo delay distribution
|
Timing-induced logic-error probability
|
Hardware-aware BNN inference
|
Classification-accuracy impact

```

The key intellectual connection is that a circuit delay is not just a circuit metric. Under a finite timing budget it becomes an error event, and that error propagates into an algorithm-level accuracy change. That circuit-to-model bridge is a compelling part of the work.

13.4 Why BNN maps well to Boolean CIM

With binary weights and activations encoded as bits, multiplication of ± 1 quantities can be represented by equality of the encoded bits. Equality is XNOR. A dot product can then be expressed using XNOR results followed by population count and a simple transform to obtain the signed accumulation.

```

bit equal    -> XNOR = 1 -> +1 product
bit different -> XNOR = 0 -> -1 product

```

If N terms are compared and P is $\text{popcount}(\text{XNOR})$,
signed dot product = $2 \cdot P - N$

Q: Why is XNOR important in your BNN context?

A: It replaces the binary multiply operation with a Boolean equality operation that maps naturally to bit-level memory-centric hardware.

13.5 Monte Carlo and variation

Monte Carlo analysis repeatedly samples statistical device/process variations and observes the resulting circuit response. In SRAM-CIM, relevant consequences include delay spread, sensing-margin variation, energy variation, and possible incorrect logic under a finite evaluation time.

Q: Why not just report average delay?

A: Average delay hides the distribution tail. Reliability depends on the probability that an operation exceeds the allowed timing window, not only the mean.

Q: What should you be able to explain from your actual simulation?

A: Which parameters or model variations were randomized, what delay metric you extracted, how you converted the delay distribution into an operation error probability, and how that probability was injected into the BNN inference model.

14. Your Current Direction: SRAM-Array Neuromorphic Computing

This section is deliberately phrased so you can discuss the direction deeply without implying that a complete accelerator has already been implemented.

14.1 What neuromorphic computing means in hardware

Neuromorphic systems borrow organizational ideas from biological neural systems: distributed memory and computation, local synaptic processing, stateful neurons, massive parallelism, and often event-driven communication. Many systems implement spiking neural networks (SNNs), where information is represented through discrete spike events and their timing/rate rather than only dense vector activations.

14.2 Basic spiking-neuron model

```
Membrane update (conceptual):
Vmem[t+1] = lambda*Vmem[t] + sum_i( w_i * S_i[t] )

If Vmem >= Vth:
    emit spike
    reset or subtract threshold
```

- $S_i[t]$ is a binary spike event.
- w_i is the synaptic weight.
- λ models leakage in a leaky integrate-and-fire neuron.
- The neuron keeps state over time, unlike a purely stateless combinational MAC.

14.3 Why SRAM is attractive for neuromorphic hardware

Strength	Why it matters
CMOS compatibility	Integrates naturally with digital neuron/control logic
Fast read/write	Useful for high event rates and online state updates
Very high endurance	Supports repeated synaptic/state access
Mature design ecosystem	Well-understood bitcells, sense amps, compilers and PVT behavior
Local weight storage	Reduces repeated movement of synaptic weights

Limitations include volatility, leakage, larger cell area than some emerging memories, and the fact that a standard SRAM cell does not natively provide a dense analog multilevel conductance.

14.4 How this differs from your BNN work

Boolean BNN SRAM-CIM	Neuromorphic SRAM direction
Primary operation: Boolean logic / XNOR	Primary concern: synaptic events, accumulation, neuron state
Dense binary inference style	Potentially event-driven / sparse activity
Circuit error -> BNN accuracy analysis	Architecture must consider temporal behavior and state
Binary weights/activations	Weights may still be binary or multibit depending on design

14.5 Event-driven energy argument - use carefully

SNNs can have sparse spike activity. If the hardware gates inactive synapses/neurons and avoids unnecessary memory/computation events, dynamic energy can fall. But you should not say that SNNs are always more efficient. Efficiency depends on event rate, coding scheme, architecture, memory access, time steps, and peripheral overhead.

Q: Why did you move from a generic MAC extension toward neuromorphic SRAM?

A: The MAC extension was the straightforward continuation of Boolean CIM, but I became more interested in the memory-compute interaction itself. Neuromorphic architectures make local storage, event-driven activity, synaptic accumulation, and state especially important, which aligns well with the memory-centric questions I want to study.

15. RRAM and Crossbar Fundamentals

Because your CV names RRAM and the employee specifically warned you to expect RRAM/crossbar questions, you need solid conceptual knowledge even though your own project is still in literature review.

15.1 What RRAM stores

Resistive RAM stores information through a device resistance or conductance state. A simple binary abstraction uses a low-resistance state (LRS) and high-resistance state (HRS). The microscopic mechanism depends on the material/device system, but many models represent a conductive filament or internal state whose evolution changes the effective resistance.

15.2 SET and RESET

Operation	Typical transition	Meaning
SET	HRS -> LRS	Device becomes more conductive
RESET	LRS -> HRS	Device becomes less conductive

Some devices are bipolar, where SET and RESET depend on voltage polarity, while others can be unipolar and depend more on magnitude/thermal effects. Do not present one switching mechanism as universal across all RRAM technologies.

15.3 1R vs 1T1R

1R crosspoint	1T1R cell
Highest density potential	Transistor increases area
Harder cell isolation	Transistor selects and current-limits the device
Sneak paths can corrupt reads/writes	Much better access control
Attractive for dense crossbars	Attractive for controlled programming and characterization

15.4 Why crossbars implement vector-matrix multiplication

```

For cell (i,j):  $I_{ij} = G_{ij} * V_i$ 
At column j:    $I_j = \sum_i (G_{ij} * V_i)$ 

Inputs  -> row voltages  $V_i$ 
Weights -> conductances  $G_{ij}$ 
Output  -> summed column current  $I_j$ 

```

Ohm's law produces the local product and Kirchhoff's current law produces the column sum. That is the physical reason a conductance crossbar is attractive for analog matrix-vector multiplication.

Q: How do you represent negative weights if conductance cannot be negative?

A: A common approach uses differential encoding, for example w proportional to $G_{plus} - G_{minus}$, using paired devices or paired columns. Other offset/reference schemes also exist.

15.5 Crossbar nonidealities

- Device-to-device and cycle-to-cycle variation.
- Limited endurance and retention concerns depending on device.
- Programming nonlinearity and difficulty hitting precise conductance levels.
- Line resistance and IR drop across large arrays.
- Sneak currents in selector-less arrays.
- ADC/DAC and peripheral energy/area overhead.
- Conductance drift, noise, and finite on/off ratio.

Q: If RRAM crossbars are mathematically elegant, why are they difficult in practice?

A: The ideal $I=GV$ and current summation are disturbed by device variation, nonlinear programming, parasitic interconnect, sensing limits and substantial peripheral-circuit overhead. System-level accuracy and efficiency depend on handling those nonidealities.

16. RRAM Compact Modeling and Verilog-A - Interview-Safe Depth

You do not need to pretend you have completed the model. You do need to understand what a useful compact model is trying to represent.

16.1 State-variable view

Electrical relation: $I = f(V, x)$
 State evolution: $dx/dt = g(V, x)$

x can represent an abstract filament length, gap, conductive fraction, or another internal state variable depending on the model.

A compact model should enforce physical or numerical boundaries on x , reproduce HRS/LRS behavior, and respond correctly to SET/RESET stimulus. More advanced models add threshold behavior, stochastic variation, temperature effects, retention, or cycle dependence.

16.2 What Verilog-A contributes

Verilog-A is an analog behavioral modeling language used to express constitutive equations, internal state and terminal currents/voltages so the device can participate in circuit simulation. It bridges device-level behavior and circuit-level exploration without requiring a full physics solver inside every transient step.

16.3 What you should say about your own project

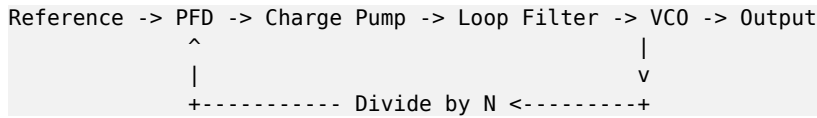
Recommended wording

I am currently at the literature-review and model-selection stage. I understand the behaviors a compact model must capture and how Verilog-A can express the current-voltage relation and state evolution, but I have not yet reached a validated implementation. My next technical step is to choose or adapt a suitable compact model and validate its switching behavior before using it in array-level neuromorphic experiments.

17. PLL and Broader VLSI Follow-Ups

Your resume also lists a 45 nm PLL, so keep a compact but technically correct explanation ready.

17.1 Charge-pump PLL block flow



- PFD compares reference and divided-feedback phase/frequency.
- Charge pump converts UP/DN information into current pulses.
- Loop filter creates a smoother control voltage and sets loop dynamics.
- VCO converts control voltage to oscillation frequency.
- Divider scales the VCO output before feedback comparison.

At ideal lock, the divided feedback frequency equals the reference frequency, so f_{VCO} is approximately $N \cdot f_{REF}$. In your project, $N=48$ and a 100 MHz reference corresponds to an ideal 4.8 GHz target; your reported simulated lock was close to that rather than exactly identical.

Q: What does lock mean?

A: The loop has driven the average phase/frequency error to a steady small value such that the divided output tracks the reference and the VCO control settles around the required operating point.

Q: Why use a loop filter?

A: It converts charge-pump pulses into a suitable control voltage and determines stability, bandwidth, damping and noise-transfer behavior of the feedback loop.

17.2 Static vs dynamic power

Dynamic power (first-order): $P_{dyn} = \alpha * C * V^2 * f$
 Static power: leakage-related power when logic is not switching

This is a useful general digital-design follow-up. Be ready to explain activity factor α , capacitance, voltage and frequency.

17.3 Fanout and critical paths

Large fanout increases effective load capacitance and can slow transitions. Buffers, replication, restructuring, or stronger cells may be used. Critical paths are the longest timing-constrained paths that limit the maximum safe clock frequency.

18. Defending Your Own Research Like an Engineer

Interviewers often probe your publication more aggressively than coursework because it reveals whether you actually owned the work. Prepare mechanism-level answers, not only an abstract summary.

18.1 Your 60-second research introduction

Suggested answer

My main research interest is memory-centric computing, particularly SRAM-based compute-in-memory and neuromorphic architectures. My published work studied a 45 nm 6T SRAM-CIM Boolean architecture and connected circuit-level timing variation to BNN inference accuracy using a hardware-aware evaluation framework. When I submitted the CV, my planned extension was a MAC-oriented SRAM-CIM accelerator. Since then, I have shifted toward SRAM-array-based neuromorphic computing because I am more interested in local synaptic computation, event-driven processing and memory-compute interaction. I am also studying RRAM for future neuromorphic hardware, but that project is currently still at the literature-review and modeling-preparation stage.

18.2 Questions you should rehearse about the SRAM-CIM paper

Question	What a strong answer must include
Why SRAM-CIM?	Data-movement motivation plus why SRAM is practical/mature
Why 6T?	Density/maturity and the constraints of using standard storage cells
How are Boolean functions generated?	Exact wordline/bitline/precharge/sensing behavior from your own schematic
Why XNOR?	Binary +/-1 multiply equivalence and popcount mapping
Why Monte Carlo?	Variation distribution and tail/error probability, not just mean delay
How did delay become a BNN error?	Finite timing window -> logic failure probability -> injected inference errors
What did you personally do?	Specific circuit, simulation, Python/modeling tasks you actually performed
Main limitation?	Discuss realistic array/peripheral/PVT/scale constraints, not vague future work

18.3 Questions that detect overclaiming

- Show me the exact path that performs XNOR.
- What happens for each input combination 00, 01, 10, 11?
- Which circuit node did you measure for delay?
- How was your timing threshold chosen?
- What distribution did the Monte Carlo results have?
- Which part of the PyTorch inference did you modify?
- How would the result change with a larger array?
- What is the peripheral circuitry and what does it cost?

Before the interview

Open your actual paper/schematic and rehearse these answers from the real implementation. Do not rely on this handbook for circuit details that depend on your specific topology.

19. Coding Drills You Should Be Able to Write Cold

Do these without autocomplete. The point is not memorizing every semicolon; it is building enough fluency that syntax does not consume your thinking time.

19.1 4:1 MUX

```
module mux4(
    input [3:0] d,
    input [1:0] s,
    output reg y
);
always @(*) begin
    case (s)
        2'b00: y = d[0];
        2'b01: y = d[1];
        2'b10: y = d[2];
        2'b11: y = d[3];
    endcase
end
endmodule
```

19.2 2-to-4 decoder with enable

```
always @(*) begin
    y = 4'b0000;
    if (en) begin
        case (a)
            2'b00: y = 4'b0001;
            2'b01: y = 4'b0010;
            2'b10: y = 4'b0100;
            2'b11: y = 4'b1000;
        endcase
    end
end
```

19.3 DFF with asynchronous reset

```
always @(posedge clk or posedge rst) begin
    if (rst) q <= 1'b0;
    else    q <= d;
end
```

Q: How would synchronous reset differ?

A: Remove reset from the sensitivity list and test reset only inside the posedge clk block. Then reset takes effect only on an active clock edge.

19.4 FIFO-like pointer update

```
always @(posedge clk) begin
    if (rst) wptr <= 0;
    else if (wr_en && !full) wptr <= wptr + 1'b1;
end
```

The interview follow-up is usually not syntax. It is: how do you compute full, what happens at wrap-around, and what changes for asynchronous clocks?

20. Mock Interview - Fundamentals Round

Try answering each aloud before reading the model answer. Keep the first answer concise; expand only when the interviewer follows up.

Q: Combinational vs sequential?

A: Combinational output depends only on current inputs. Sequential logic contains state, so its behavior depends on current inputs plus stored history. Synchronous sequential circuits usually store state in edge-triggered flip-flops.

Q: What causes an inferred latch?

A: A procedural combinational block fails to assign an output on every possible path, so hardware must preserve the old value for uncovered conditions.

Q: Blocking vs nonblocking?

A: Use blocking assignments for combinational procedural logic and nonblocking assignments for clocked state. Nonblocking updates model simultaneous register behavior.

Q: Can slowing the clock fix hold?

A: Normally no. Hold is a minimum-delay requirement around the same capture edge; you fix it by increasing minimum data-path delay or adjusting clocking, not by extending the next period.

Q: Can a 2-FF synchronizer synchronize a 16-bit bus?

A: Not safely bit-by-bit if the bus can change freely. Different bits can be captured from different source states. Use a handshake for held data or an async FIFO for streaming data.

Q: Why Gray code in async FIFO?

A: Adjacent Gray pointer values differ by one bit, which reduces multi-bit transition ambiguity when the pointer crosses an asynchronous clock boundary. The changing bit is still synchronized.

Q: When does ready-valid transfer occur?

A: On the active clock edge where both ready and valid are high.

Q: What should the sender do when valid=1 and ready=0?

A: Hold the current data item and valid stable until it is accepted, unless the specific protocol defines a different behavior.

Q: Moore vs Mealy?

A: Moore output depends only on state; Mealy output depends on state and current input. Mealy can respond with fewer states but its combinational outputs require more timing care.

Q: Why does a FIFO need an extra wrap bit?

A: Because equal memory-address bits can mean either empty or full after wrap-around. The extra phase bit distinguishes whether the write pointer has lapped the read pointer.

21. Mock Interview - Research and Resume Round

Q: Why did you choose SRAM instead of RRAM for your published CIM work?

A: SRAM is a mature CMOS memory with fast operation, high endurance and predictable integration, which makes it suitable for studying logic-in-memory and variation-aware circuit behavior without adding the device-level uncertainty of an emerging memory. RRAM is attractive for density and analog conductance, but it brings different nonidealities.

Q: What is the main benefit of compute-in-memory?

A: Reducing data movement between storage and compute. The arithmetic itself may be simple, but moving large numbers of weights and activations repeatedly is expensive in energy and latency.

Q: How does circuit delay become an inference error?

A: If the circuit must be evaluated within a fixed timing window, a variation-induced delay exceeding that window can produce an incorrect or unresolved Boolean result. The probability of that event can be represented as an operation error rate and injected into the BNN computation.

Q: Why is average accuracy over Monte Carlo runs not enough?

A: The spread matters. Reporting mean, standard deviation, and tail behavior shows robustness and whether rare circuit/error patterns produce significant system-level degradation.

Q: What is different about neuromorphic computing?

A: It emphasizes local state, synaptic processing and often event-driven temporal computation. In an SNN, neurons integrate spike events over time and emit spikes after crossing thresholds, so the hardware problem includes state update and sparse event handling, not only dense MAC operations.

Q: Why SRAM for neuromorphic synapses?

A: It is fast, highly enduring, CMOS-compatible and can keep weights close to compute. Its limitations are area, volatility, leakage and lack of native dense analog multilevel conductance.

Q: Why RRAM crossbars for neural workloads?

A: A device conductance can represent a weight. Applying row voltages produces $I=GV$ at each crosspoint and column currents sum by Kirchhoff's law, giving analog vector-matrix multiplication in the array.

Q: What is a major crossbar problem?

A: Nonidealities such as device variation, IR drop, sneak paths, programming precision and ADC/DAC overhead can reduce accuracy and efficiency, especially as arrays scale.

Q: What exactly have you done in RRAM?

A: My current progress is literature review and compact-model study. I understand the modeling objectives and device/crossbar behavior, but I have not yet completed or validated a Verilog-A model.

22. Whiteboard Problem-Solving Drills

Do not just read these. Put a timer on and solve them on paper.

11. Draw an 8:1 MUX using only 2:1 MUXes. Label select signals and compute the number of stages.
12. Design a 4-to-2 priority encoder with D3 highest priority. Produce truth behavior, equations, and a MUX-based structure.
13. Draw a register-to-register path and derive setup and hold constraints using your chosen skew sign convention.
14. A 1-cycle pulse crosses from 250 MHz to 40 MHz. Explain why a 2-FF synchronizer might miss it and propose two safe methods.
15. Design a synchronous FIFO of depth 16. What pointer width do you use? How do you detect empty and full?
16. Explain an asynchronous FIFO of depth 8, including binary pointers, Gray pointers, 2-FF synchronizers, and local flag generation.
17. Write a 4-bit up/down counter with enable and synchronous reset.
18. Write a 1011 overlapping sequence detector as a Moore machine, then explain how a Mealy version differs.
19. Draw a 6T SRAM cell and explain hold, read, write, read disturb, and the read/write sizing tradeoff.
20. Draw a simple RRAM crossbar and derive $I_j = \sum(V_i * G_{ij})$. Then explain how you would represent negative weights.

22.1 Interview reasoning pattern

Use this when you get stuck

1) Restate the requirement. 2) State assumptions. 3) Draw the simplest block diagram. 4) Identify state/clock domains. 5) Derive the condition or equation. 6) Check corner cases. 7) Only then write RTL.

23. Tricky Follow-Up Questions

Question	Core idea you should mention
Why is casex dangerous?	X values can be treated as don't-cares and hide real bugs.
What is FIFO flag pessimism?	Async synchronization delays remote pointer visibility; flags may assert/deassert conservatively.
Can Gray code jump by more than one bit?	Adjacent binary counts map to one-bit Gray changes, but the destination may skip intermediate counts if clocks differ; sampled values can be non-adjacent in time.
Why can Mealy outputs glitch?	They depend combinationaly on inputs, which may change within the cycle.
What happens if ready is combinationaly dependent on valid and vice versa across modules?	Can create a combinational loop or timing problem; interface design should avoid circular dependence.
Why precharge SRAM bitlines?	Provides a known common starting point so small cell-dependent discharge differences can be sensed quickly.
Why not wait for a full rail-to-rail bitline swing?	It would be much slower and consume more dynamic energy; sense amps detect small differentials.
What limits analog CIM accuracy?	Variation, noise, nonlinearities, parasitics, finite precision, converter accuracy.
What does sparsity buy in neuromorphic hardware?	If exploited architecturally, inactive events avoid switching, memory access or accumulation.
Why can RRAM need current compliance?	SET can otherwise create excessive conductive filament/current and damage or over-program the device; selector transistor can help limit current.

24. Focused Preparation Plan

Use the schedule below as a compressed plan. If you have more days, spread the same modules out and add repetition rather than continuously adding new topics.

Session	Focus	Deliverable
1	Digital basics + MUX/encoder/counter/register	Draw/code each from memory
2	Setup/hold/skew + metastability	Explain equations and fixes aloud
3	CDC + 2FF + pulse/bus crossing	Choose safe architecture for scenarios
4	Synchronous + asynchronous FIFO	Draw complete async FIFO without notes
5	Ready-valid + RTL + testbench	Write 3 small modules + TB
6	FSM + sequence detector	Build Moore and Mealy examples
7	6T/8T SRAM fundamentals	Draw cell and explain hold/read/write/SNM
8	Your CIM paper	Review real schematics and methodology
9	Neuromorphic SRAM + RRAM crossbars	Give 2-minute explanations
10	Full mock interview	Answer under time pressure; review weak spots

24.1 Night-before checklist

- Can draw 8:1 MUX using seven 2:1 MUXes.
- Can explain setup vs hold and fixes without mixing them up.
- Can explain metastability and why 2-FF only suits certain signals.
- Can draw asynchronous FIFO with Gray pointers and synchronizers.
- Can explain full/empty conceptually and the wrap-bit idea.
- Can write counter, encoder/decoder, simple FSM, and testbench from memory.
- Can explain ready-valid/backpressure in under 30 seconds.
- Can draw 6T SRAM and explain read/write/read disturb.
- Can explain your exact SRAM-CIM circuit using your actual schematic.
- Can state your MAC-to-neuromorphic change and RRAM status clearly and honestly.

25. Last-Minute Cheat Sheet

Topic	One-line memory trigger
Combinational	output = $f(\text{current inputs})$
Sequential	output/state depends on stored history
Latch	level-sensitive; missing combinational assignment can infer one
DFF	edge-triggered state storage; $Q_{\text{next}}=D$
Blocking	combinational procedural =
Nonblocking	clocked state $\leq$
Setup	data too late $\rightarrow$ reduce max delay / increase period
Hold	data too early $\rightarrow$ increase min delay; frequency usually not the fix
Metastability	cannot eliminate; contain and increase resolution time
2-FF sync	single-bit level CDC, latency acceptable
Multi-bit CDC	handshake or async FIFO, not independent 2FFs
Gray code	one-bit transition between adjacent counts
FIFO empty	reader caught writer
FIFO full	writer one full buffer ahead of reader
Ready-valid	transfer when both are 1 on edge
Backpressure	ready=0 stalls/holds producer item
Moore	output= $f(\text{state})$
Mealy	output= $f(\text{state}, \text{input})$
6T SRAM	2 cross-coupled inverters + 2 access transistors
Read disturb	read connection perturbs internal 0 node
SNM	DC noise tolerance of bistable cell
CIM	reduce memory-compute data movement
BNN XNOR	equality encodes binary ± 1 multiplication
Monte Carlo	statistical variation $\rightarrow$ response distribution/tail risk
SNN neuron	integrate weighted spikes $\rightarrow$ threshold $\rightarrow$ spike/reset
RRAM	conductance/resistance state; HRS/LRS; SET/RESET
Crossbar VMM	$I_j = \sum(V_i * G_{ij})$
1T1R	transistor gives cell selection/current control

25.1 Five answers to keep polished

21. Tell me about yourself / your research direction.
22. Explain asynchronous FIFO and why Gray code is used.
23. Explain setup and hold violations and how you fix each.
24. Explain your SRAM-CIM paper from transistor/circuit level to BNN accuracy.
25. Why does your CV say MAC/RRAM work when your current status is different?

Final mindset

The interview is not a closed-book exam where every obscure fact must be memorized. The strongest signal is structured reasoning: state assumptions, draw the architecture, explain the physical/digital mechanism, identify corner cases, and be exact about what you personally implemented.

Appendix A. 80 Rapid-Fire Questions

1. What is the difference between combinational and sequential logic?
2. Latch vs flip-flop?
3. What is setup time?
4. What is hold time?
5. What is positive clock skew?
6. How do you fix setup?
7. How do you fix hold?
8. Can lower frequency fix hold?
9. What is metastability?
10. Can metastability be eliminated?
11. Why use two synchronizer FFs?
12. When is a 2-FF synchronizer appropriate?
13. Why is bit-by-bit multi-bit synchronization unsafe?
14. How do you transfer a slow-changing bus?
15. How do you transfer a stream between unrelated clocks?
16. What is Gray code?
17. Binary-to-Gray equation?
18. Why Gray pointers in FIFO?
19. What is FIFO overflow?
20. What is FIFO underflow?
21. Why add a wrap bit to FIFO pointers?
22. How is synchronous FIFO full detected?
23. How is async FIFO empty generated?
24. Why is full local to write clock?
25. Why is empty local to read clock?
26. What is backpressure?
27. When does ready-valid transfer?
28. What must data do while valid=1 ready=0?
29. What is a combinational loop in handshake logic?
30. What is skid buffering conceptually?
31. Blocking vs nonblocking?
32. What does always @(*) mean?
33. initial vs always?
34. wire vs reg?
35. logic vs wire/reg?
36. What causes latch inference?
37. Why use default in case?
38. casez vs casex?
39. What is parameterization?
40. What is sign extension?
41. What is a synchronous reset?
42. What is an asynchronous reset?
43. What is async assert / sync deassert?
44. What is a Moore FSM?
45. What is a Mealy FSM?
46. What is overlapping sequence detection?
47. How many 2:1 MUXes for an 8:1 MUX?
48. How many stages?
49. Encoder vs priority encoder?
50. Decoder vs demux?
51. What is dynamic power?
52. What increases dynamic power?
53. What is fanout?
54. What is a critical path?
55. Why pipeline?

56. What is 6T SRAM?
57. How does SRAM hold data?
58. How is SRAM read?
59. How is SRAM written?
60. What is read disturb?
61. What is SNM?
62. Why can 8T improve read stability?
63. Why precharge bitlines?
64. Why use a sense amplifier?
65. What is compute-in-memory?
66. Why is data movement expensive?
67. Why XNOR in BNN?
68. What is popcount?
69. Why Monte Carlo?
70. Why does tail delay matter?
71. What is neuromorphic computing?
72. What is an SNN?
73. What does a leaky integrate-and-fire neuron do?
74. Why can event sparsity save energy?
75. Why SRAM for neuromorphic computing?
76. What is RRAM?
77. HRS vs LRS?
78. SET vs RESET?
79. Why RRAM crossbars perform VMM?
80. What is a sneak path?

Appendix B. Questions to Ask the Interviewer

Good questions should show curiosity about the engineering work without sounding as if you only care about titles or perks.

- For fresh engineers, which part of the digital design flow do new team members usually start with: RTL development, verification, integration, synthesis, or something else?
- What kinds of design blocks or interfaces are most common in the team?
- How closely do RTL designers interact with verification and physical-design teams during timing closure and debug?
- What technical skills distinguish people who ramp up quickly in the first six months?
- Does the team work more with Verilog/SystemVerilog RTL, scripting/automation, or a combination?
- How are design reviews and verification sign-off typically organized?

Avoid asking all of them. Pick one or two based on how the conversation went.

Appendix C. Personal Pre-Interview Notes

Use this page to write the exact details from your own SRAM-CIM schematic and paper that cannot be safely generalized in a handbook.

Exact XNOR mechanism / signal sequence:

Exact Monte Carlo variables:

Timing-error threshold definition:

Your personal contribution:

Most important result:

Main limitation:

Neuromorphic next experiment:

RRAM next technical step:
